i uploaded a image but i don't know the tentative questions to ask about the image for imagejev.How can i resolve that?

You do **not** need to know the questions in advance. Start with a generic “image triage” question set, inspect a few responses, then replace or refine questions based on what the image actually is and what decision your workflow needs. ImageJev is built for focused decisions: you provide an image, a question, and the allowed answers; it returns structured probabilities and confidence rather than open-ended text. [1]

Use a universal first pass

For any uploaded document or photo, ask these four starter questions. They are broad enough to work before you know whether it is an invoice, receipt, screenshot, form, or something else.

Question	Type	Options / scale	Why ask it
What is the main type of image?	`choice`	`invoice`, `receipt`, `bank_statement`, `form`, `screenshot`, `photo`, `other`	Determines the downstream workflow
Is the image readable enough for processing?	`choice`	`yes`, `partially`, `no`	Routes poor-quality uploads back to the user
Does the image contain financial or transaction information?	`noul`	Yes/no	Flags whether it belongs in a finance workflow
How usable is the image for automated review?	`score`	1 to 5	Gives a quality signal for prioritization

ImageJev supports exactly these three decision types: `choice` for selecting from options, `noul` for a yes/no visual statement, and `score` for an ordered rating scale. [2] [1]

Starter Python questions

```
starter_questions = {
    "image_type": {
        "type": "choice",
        "instructions": "What is the main type of image?",
        "criteria": {
            "invoice": "A supplier invoice requesting payment",
            "receipt": "A purchase or expense receipt",
            "bank_statement": "A bank account statement",
            "form": "A structured form or application",
            "screenshot": "A screenshot of software or a website",
```

```
            "photo": "A general photograph",
            "other": "None of the above"
        }
    },
    "readable": {
        "type": "choice",
        "instructions": "Is the image readable enough for automated processing?",
        "criteria": {
            "yes": "Key information is legible",
            "partially": "Some information is unclear or cropped",
            "no": "The image is too blurry, dark, rotated, or incomplete"
        }
    },
    "financial_document": {
        "type": "noul",
        "instructions": "The image contains financial or transaction information."
    },
    "usability": {
        "type": "score",
        "instructions": "Rate how usable the image is for automated review.",
        "criteria": [
            "Unusable",
            "Poor",
            "Fair",
            "Good",
            "Excellent"
        ]
    }
}
```

The API returns the selected `choice`, `score`, probabilities, confidence, and—importantly—an `unknown_probability`/abstention signal, so your app can avoid forcing a guess. [3] [1]

Then branch by image type

After the first response, use the predicted `image_type` to ask a second, more specific set of questions.

If it is a receipt

```
receipt_questions = {
    "has_total": {
        "type": "noul",
        "instructions": "The image visibly shows a total amount."
    },
    "has_date": {
        "type": "noul",
        "instructions": "The image visibly shows a transaction date."
    },
    "has_merchant": {
        "type": "noul",
        "instructions": "The image visibly shows a merchant or store name."
    },
    "expense_category": {
```

```
        "type": "choice",
        "instructions": "Which expense category best fits this receipt?",
        "criteria": {
            "meals": None,
            "travel": None,
            "office_supplies": None,
            "software": None,
            "other": None
        }
    }
}
```

If it is an invoice

```
invoice_questions = {
    "has_invoice_number": {
        "type": "noul",
        "instructions": "The image visibly shows an invoice number."
    },
    "has_due_date": {
        "type": "noul",
        "instructions": "The image visibly shows a payment due date."
    },
    "has_total": {
        "type": "noul",
        "instructions": "The image visibly shows a total amount due."
    },
    "payment_status": {
        "type": "choice",
        "instructions": "What is the apparent payment status?",
        "criteria": {
            "unpaid": None,
            "paid": None,
            "partially_paid": None,
            "unknown": None
        }
    }
}
```

If it is neither

```
fallback_questions = {
    "contains_text": {
        "type": "noul",
        "instructions": "The image contains meaningful text."
    },
    "contains_table": {
        "type": "noul",
        "instructions": "The image contains a table or structured rows of informatio
    },
    "recommended_team": {
        "type": "choice",
        "instructions": "Which team should review this document?",
```

```
        "criteria": {
            "finance": None,
            "procurement": None,
            "operations": None,
            "support": None,
            "other": None
        }
    }
}
```

A practical discovery loop

1. **Upload one representative image.**
2. **Run only the four starter questions.**
3. **Read the predicted image type and confidence.**
4. **Open the image yourself and write down the 3–5 facts a human reviewer would check.**
5. **Convert each fact into a `noul`, `choice`, or `score` question.**
6. **Run the refined questions on 10–20 images.**
7. **Keep questions that consistently produce useful, confident answers; remove or reword ones that often return `unknown`.**

For example, if the first pass says `receipt`, a human reviewer would usually check: merchant, date, total, payment method, and whether it matches the claimed expense. Those become your second-stage questions.

Keep questions focused

Do not ask one broad question such as “Is this a valid expense receipt?” Break it into independent checks: document type, readability, merchant present, date present, total present, and context match. ImageJev’s documentation explicitly recommends asking independent checks separately and combining their results in your application logic. [1]

A good rule is:

- Use `noul` for one clear visual fact: “A total amount is visible.”
- Use `choice` when there are mutually exclusive categories: “invoice / receipt / other.”
- Use `score` for graded judgments: image quality, damage severity, or urgency.
- Add an explicit `unknown` option whenever evidence may be missing. [2] [1]

For your invoice bot

For the Invoice and Receipt Intake Bot, begin with the universal set above. Once the image is classified as a receipt or invoice, ask the matching second-stage questions, then route using rules such as:

- `readable == no` → request a better image.
- `image_type == receipt` and total/date/merchant checks pass → expenses queue.

- `image_type == invoice` and invoice number/total are visible → accounts payable queue.
- Any `unknown` or low-confidence result → manual review.

This two-stage approach solves your problem: you do not need to predict every question before seeing the image; you first discover what it is, then ask only the questions relevant to that document type. [1] [2]

⁂

1. https://imagejev.mintlify.app/introduction
2. https://github.com/nomadoor/ComfyUI-TypedDecision
3. https://www.jevdirectory.org/resources/mohit67890-imajev
4. https://github.com/nomadoor/ComfyUI-TypedDecision/blob/main/README.md
5. https://github.com/mohit67890/imajev/blob/main/scripts/playground/README.md
6. https://github.com/mohit67890/imajev
7. https://mohit67890.github.io/imajev/report/
8. https://github.com/fstandhartinger/jevbench/issues/110
9. https://github.com/mohit67890/imajev/blob/main/docs/imajev-bench-v2-overview.md
10. https://arxiv.org/html/2610.19118v1
11. https://github.com/AnotiaWang/awesome-decision-models/blob/main/README.md
12. https://huggingface.co/Jwuthrich/selfjev-4b
13. https://github.com/AnotiaWang/awesome-decision-models?ref=jevdirectory.org
14. https://huggingface.co/Jwuthrich/selfjev-4b-vision
15. https://pypi.org/project/imagejev/